```
-          onClick={changeState}>
-          {seatNumber + 1}
-        </span>
-      </td>
40:   )

-   }
-
-   export default Seat
```

The first new React-specific line here is line 12, `const [status, setStatus] = React.useState(props.initialStatus)`. We are calling the React method `useState`, which is a React *hook* method. It's called a hook because it allows our component to "hook into" the React rendering life cycle to allow the component to change the larger system. React defines several different default hooks, plus you can create your own.

Right here, right now, we're calling `useState`. What `useState` does is register a given value as being a part of React state such that changing that value triggers a re-render. The argument to `useState` is the initial value of the new state object in question—in our case, we're taking the value from an `initialState` passed in as a prop. (We'll need to change the `row.tsx` component so that its call looks like this: `<Seat key={seatNumber} seatNumber={seatNumber} initialStatus="unsold" />`.)

The `useState` method has kind of a weird return value; it returns a two-element array, which you typically capture into two different variables using JavaScript's destructuring syntax. Here we are capturing the values into variables named `status` and `setStatus`. The first return value, in our case, `status`, is a property that has the current value of our state. The second return value, `setStatus`, is our state setter—a function that we can call later in our component to change the value of the state and trigger a redraw.

The useState Hook Initial Value